

Physical Design

Counter

Name	***Sidra Naveed***
Instructor	*Hira Sohail*
Date	*10th October 2025*

Objective

The aim of this lab is to use the generated netlist and transfer it to the physical layout by using Cadence (Innovus). After importing the design to Innovus for back-end design, steps for floor planning and power planning all the way to GDSII will be performed.

After completing this lab, I am able to

- Transfer the netlist into physical layout.
- Complete floor planning, power planning, pins assignment, placement, routing, CTS, optimization and DRC LVS.
- Generating complete GDSII

Introduction

Importing a design into Innovus, which is Cadence's digital implementation tool, involves the process of bringing in a netlist or design description into the tool for further physical design activities. Once the design is imported, floor planning and power planning are crucial steps in the physical design flow. Other steps include placement of standard cells, routing, CTS, nano routing and finally GDS formation after clearing DRC and LVS.

Theory of Concept

When the design is imported the following files are given.

Field	Description
Verilog files	Contain the names of gate-level Verilog netlist files.
LEF files	Library of components and physical data for the components in LEF format (also contains routing layers and DRC rules).
MMMC View Definition file, *counter.view*	Contains pointers to timing libraries, technology files for extraction, and SDC constraints files.

Floor planning is the initial step in physical design where you define the approximate locations of major components (blocks) within the chip. This includes the placement of macros, standard cells, and other important elements. The goal of floor planning is to meet design constraints such as area utilization, timing, and routability. It provides a high-level view of how the different components of the design will be organized on the chip.

After floor planning, the step of pin assignment according to the design is performed.

Power planning involves designing the power distribution network to ensure that each part of the chip receives the required power. This includes creating power grids, distributing power and ground connections, and minimizing voltage drops.

Placement: Assign locations for each logic cell and other components on the chip. Optimize

placement for factors like wirelength, timing, and congestion.

Clock Tree Synthesis (CTS): Build a clock distribution network to provide a stable and synchronized clock to all sequential elements.

Optimize for skew, balancing, and power.

Routing: Connect the pins of logic cells and other components using metal layers. Resolve

congestion and optimize for wirelength, parasitics, and timing.

Optimization: Perform various optimization techniques to improve the design metrics. Timing optimization to meet the required performance.

Power optimization to reduce overall power consumption.

Static Timing Analysis (STA): Analyze the timing characteristics of the design to ensure that it meets the required performance.

Identify and fix timing violations.

Back-end Design Demo:

- Create the folder by the name of Backend in your current directory.
- You must have the following files in your working directory.

Pin Assignment file:

- Inside the provided folder, we have lib, gds, mmmc and lef files that contain information about different process of standard cell as well as the physical information of each cell of standard cell library required for back-end design.
- .io contains the information of pin direction, io file for counter is given.

```
############################################################
# IO Assignment File (Version 2)
# For: counter(clk, reset, count[3:0])
# Notes:
#   - "version: 2" → Cadence format for IO pin placement
#   - "offset"     → minimum distance of the *first* pin
#                    from the corresponding corner (µm)
#   - "Pin:"       → format = Pin: <name> <edge> <metal layer>
#   - Edges: N = North, E = East, S = South, W = West
#   - Distances are measured from the nearest corner
#     along that edge (in microns).
############################################################
# Kindly view this file and try by giving different values of offset
version: 2

# ---------- North Edge ----------
offset: 2
Pin: clk       N 1        ;# clk at 2µm from NW corner

offset: 4
Pin: count[0]  N 1        ;# count[0] at 4µm from NW corner

# ---------- East Edge ----------
offset: 3
Pin: reset     E 1        ;# reset at 3µm from NE corner

# ---------- South Edge ----------
offset: 2
Pin: count[1]  S 1        ;# count[1] at 2µm from SE corner

offset: 4
Pin: count[2]  S 1        ;# count[2] at 4µm from SE corner

# ---------- West Edge ----------
offset: 3
Pin: count[3]  W 1        ;# count[3] at 3µm from SW corner
```

For innovus, open the terminal, and write the following commands:

Csh

Source cshrc

innovus\

```
[cc@ncdc-0043 Sidra_backend_final]$ csh
[cc@ncdc-0043 Sidra_backend_final]$ source cshrc
**************************************************
******        Welcome to Cadence Tools         *******
**************************************************

[cc@ncdc-0043 Sidra_backend_final]$ innovus

Cadence Innovus(TM) Implementation System.
Copyright 2021 Cadence Design Systems, Inc. All rights reserved worldwide.

Version:        v21.18-s099_1, built Tue Jul 18 13:03:50 PDT 2023
Options:
Date:           Tue Oct  7 16:20:24 2025
Host:           ncdc-0043 (x86_64 w/Linux 4.18.0-372.32.1.el8_6.x86_64) (12cores*20cpus*12th Gen Intel(R) Core(TM) i7-12700 25600KB)
OS:             Red Hat Enterprise Linux 8.6 (Ootpa)

License:
                [16:20:24.301433] Configured Lic search path (21.01-s002): 5280@10.19.32.90

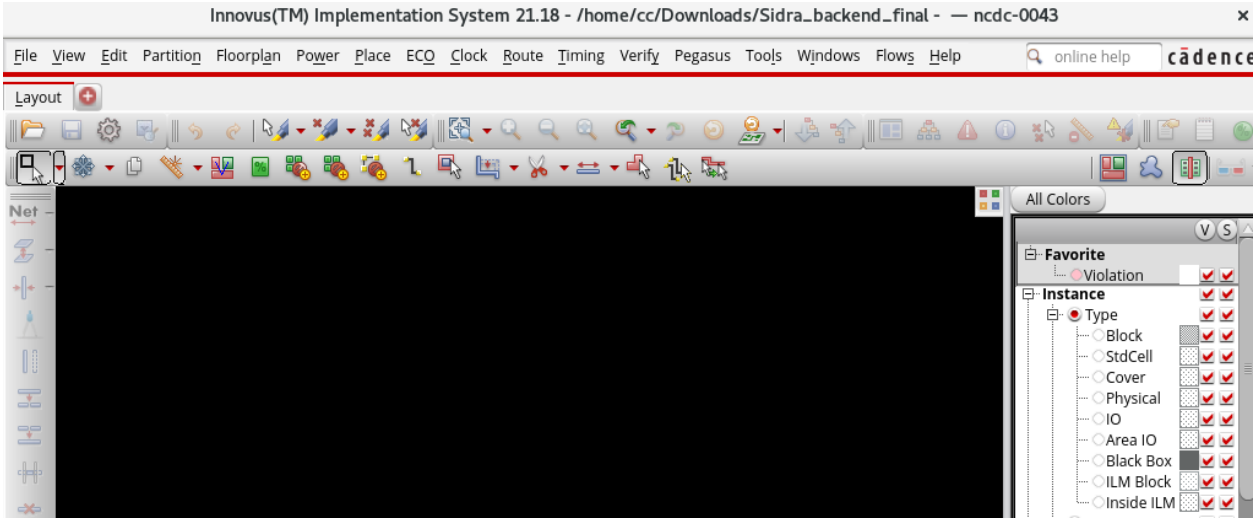
                invs    Innovus Implementation System   21.1    checkout succeeded
                8 CPU jobs allowed with the current license(s). Use setMultiCpuUsage to set your required CPU count.
Create and set the environment variable TMPDIR to /tmp/innovus_temp_18723_ncdc-0043_cc_zfZBQi.

Change the soft stacksize limit to 0.2%RAM (127 mbytes). Set global soft_stack_size_limit to change the value.

**INFO:  MMMC transition support version v31-84

[INFO] Loading Pegasus 23.13 fill procedures
innovus 1>
```

This will invoke innovus i.e tool for backend design in Cadence.



Note:

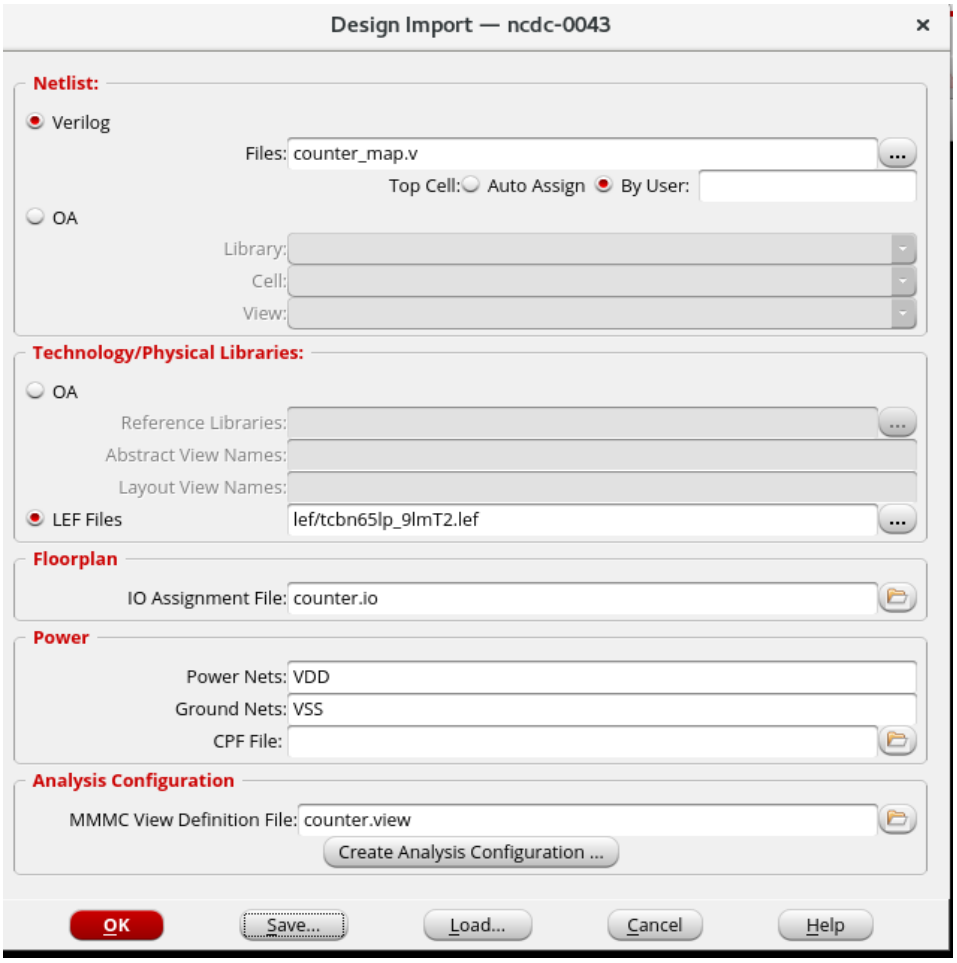
Here I have updated the paths of different files.

```
create_rc_corner -name RC_TC -cap_table /home/cc/Downloads/Sidra_backend_final/mmmc_files/captable/cln65lp_1p09m+alrdl_top2_typical.captable
create_rc_corner -name RC_WC -cap_table /home/cc/Downloads/Sidra_backend_final/mmmc_files/captable/cln65lp_1p09m+alrdl_top2_rcworst.captable
create_rc_corner -name RC_BC -cap_table /home/cc/Downloads/Sidra_backend_final/mmmc_files/captable/cln65lp_1p09m+alrdl_top2_rcbest.captable
```

```
"
# Multiple constraint modes can be defined (e.g., functional vs. test).
# Here, we define a single constraint mode for the counter design.
# Make sure about the path of sdc
create_constraint_mode -name constraints -sdc_files {/home/cc/Downloads/Sidra_backend_final/counter.sdc}
```

```
# These timing libraries are provided by the foundry and must match the design's PVT conditions.
# Make sure that your lib path is different in other PC's
create_library_set -name TC_LIBSET -timing /home/cc/Downloads/Sidra_backend_final/mmmc_files/libs/tcbn65lptc.lib
create_library_set -name WC_LIBSET -timing /home/cc/Downloads/Sidra_backend_final/mmmc_files/libs/tcbn65lpwc.lib
create_library_set -name BC_LIBSET -timing /home/cc/Downloads/Sidra_backend_final/mmmc_files/libs/tcbn65lpbc.lib
```

- Click on File => Import design, below figure will appear:



Here I have imported my files, the netlist created from frontedn of counter, library files and defined power and ground nets.

- Netlist
- Lef file
- MMMC file

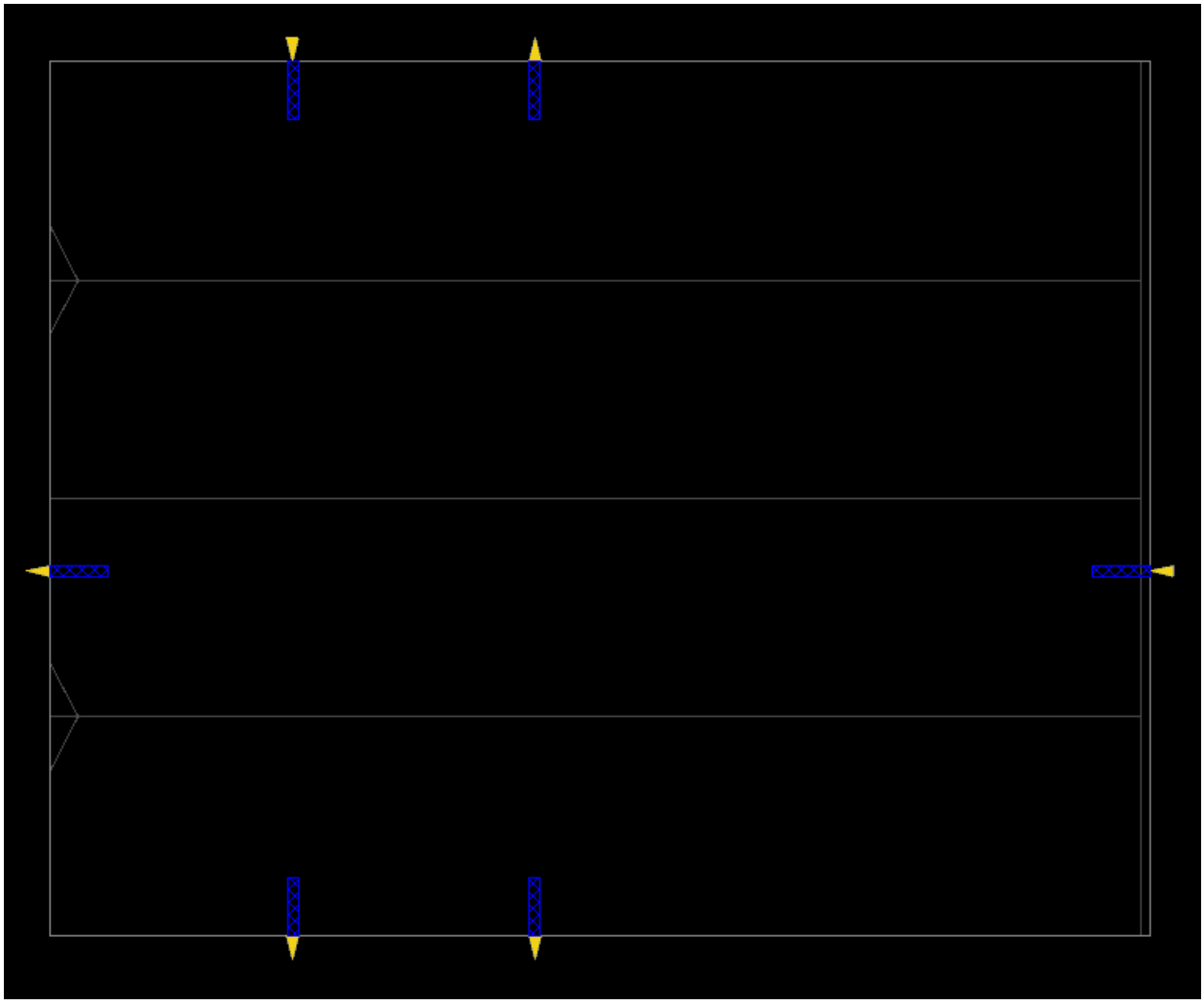
Note:

If you are facing any error:

> In a legacy environment, you cannot run init_design command. mmmc.view file is version:1.0, we need to change it to 2. Go to terminal and set the mmc version to 2 by writing following command in terminal.

set init_mmmc_version 2.

This view is shown.



Explanation:

Blue Arrows (I/O Pins)

Represent the input and output pins defined in the .io file (such as clk, reset, and out[3:0]). Their positions along the chip boundary indicate where external connections enter or leave the design.

Large Rectangle (Core Area)

Denotes the core region of the chip where all the standard cells (logic gates) will be placed.

Outer Rectangle (Die Outline)

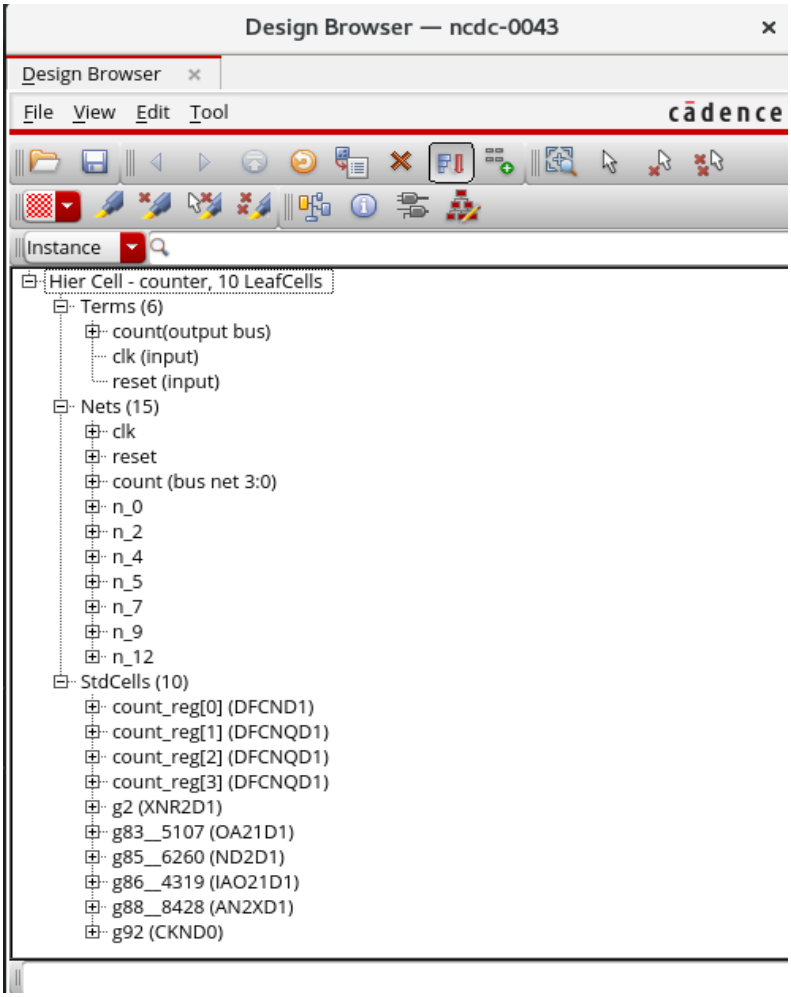
Represents the overall chip boundary. The area between the die boundary and the core area serves as the I/O and routing margin.

Horizontal Stripes (Standard Cell Rows)

Indicate the placement rows where standard cells will be arranged during the placement stage of the physical design flow.

Viewing the Design Hierarchy:

Go to tools – Design browser. Look at the IOs, nets and std cells.



Explanation:

This design hierarchy represents the logical structure of my counter circuit after importing it into Innovus. It shows that the top-level cell is named “counter”, which includes all the main connections and standard cells used in the design. The Terms section lists the input and output signals such as clk, reset, and the 4-bit output bus count[3:0]. The Nets section displays how these signals are connected internally through different wires or interconnections. The StdCells section lists the actual standard cells (like flip-flops and logic gates) that implement the counter’s functionality. For example, count_reg[0–3] are D flip-flops used to store the counter bits, while the remaining gates (AND, OR, NAND, etc.) perform logical operations required for counting. Overall, this hierarchy shows the detailed internal structure and connectivity of my counter design

Floor planning:

It is a critical stage in the physical design flow where you define the physical organization and layout of the entire chip before placement. It sets the blueprint for how standard cells, macros, IO pads, and power structures will be arranged on the silicon die.

It defines:

- Core area
- Die area
- Macros/IPs
- IO pins
- Core utilization
- Aspect ratio

Click on Floorplan - specify Floorplan below figure will be open

Check the Dimension, For this I read my area report counter. During Front end my core was 45µm2

```
============================================================
  Generated by:           Genus(TM) Synthesis Solution 21.18-s082_1
  Generated on:           Oct 01 2025  12:16:33 pm
  Module:                 counter
  Operating conditions:   NCCOM (balanced_tree)
  Wireload mode:          segmented
  Area mode:              timing library
============================================================

Instance Module  Cell Count  Cell Area  Net Area   Total Area      Wireload
-----------------------------------------------------------------------------
counter                  10      45.720     0.000       45.720 ZeroWireload (S)
```

Core Utilization:

It is recommended to keep core utilization as 60 - 70%. Core utilization is basically the area occupied by standard cells. We can check from the area report generated by genus after synthesis as above that

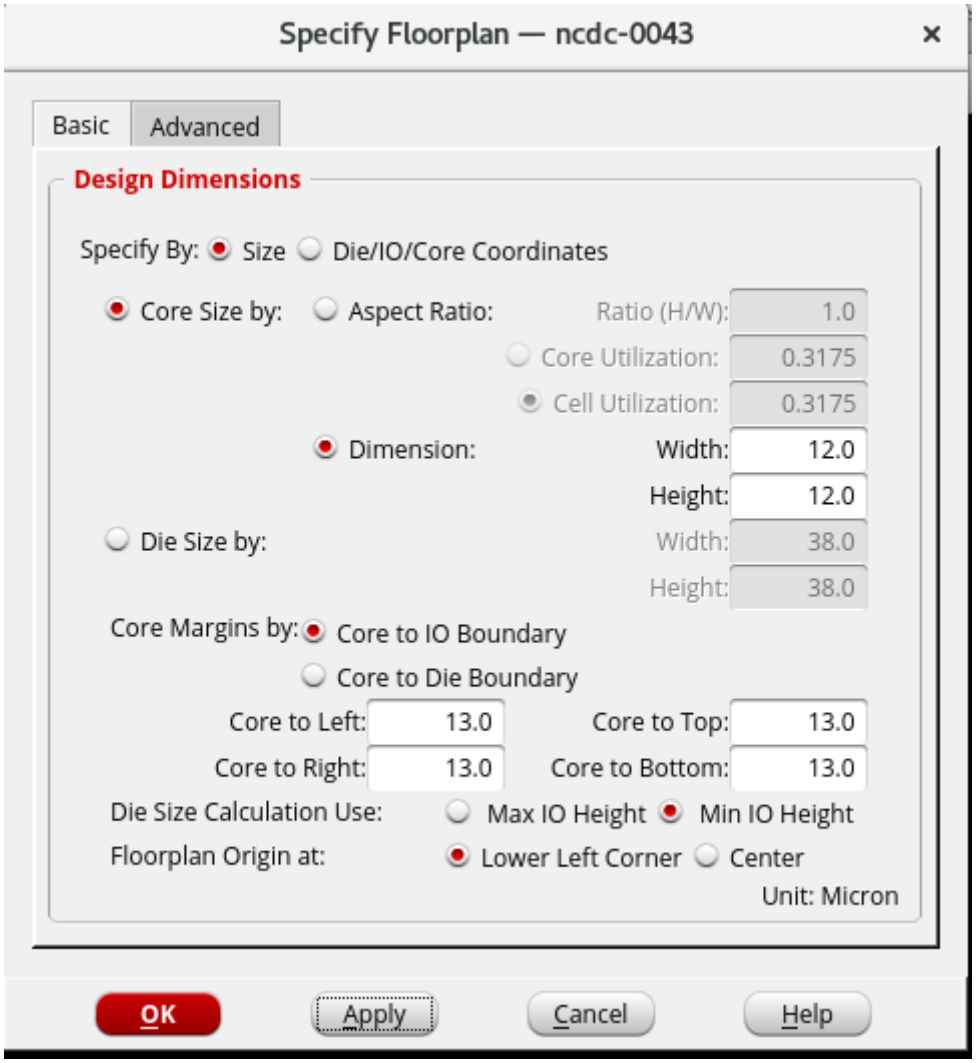
total area = 45 µm2.

To find out Required core area, use formula:

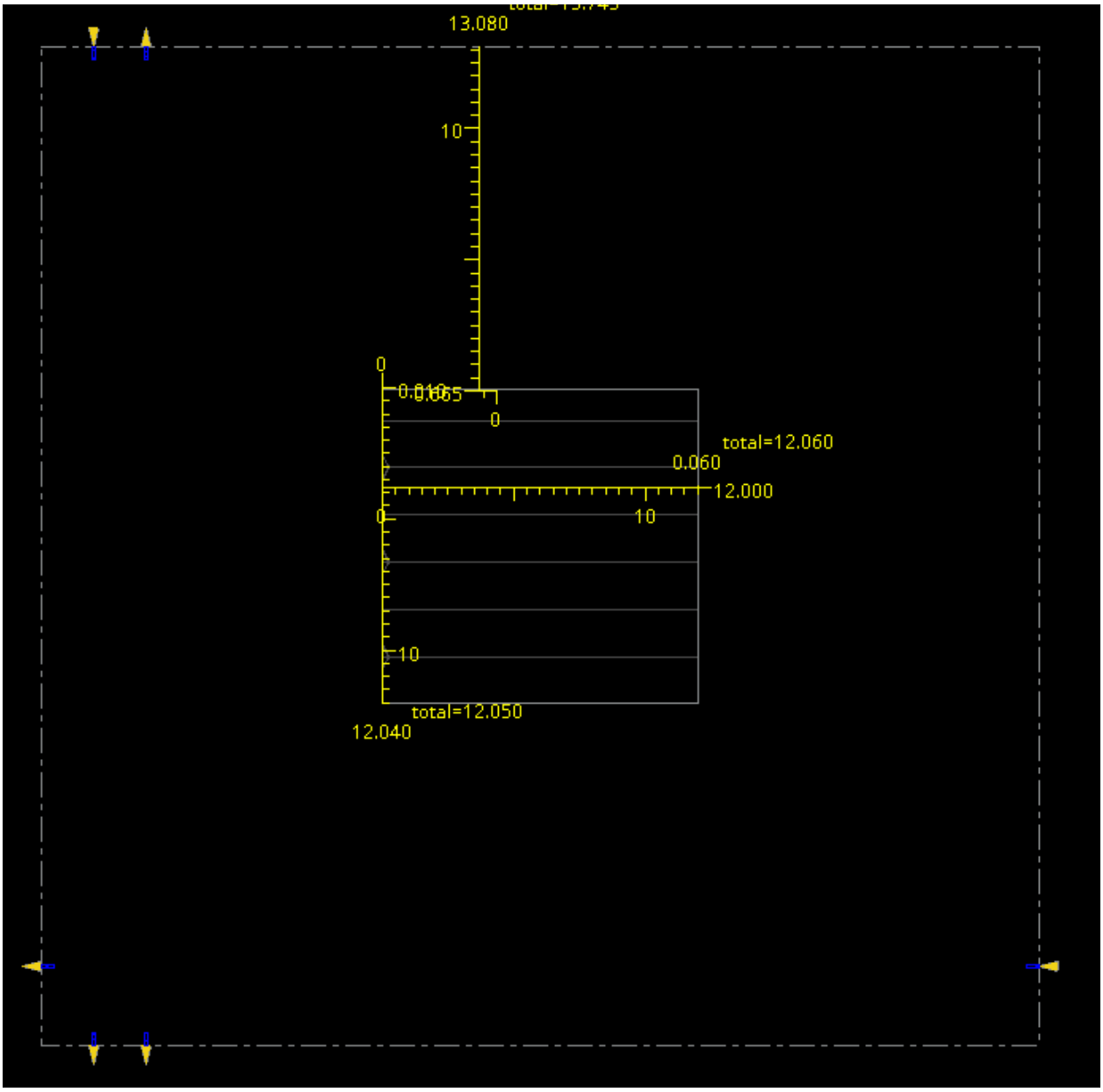
Required core area = 45/0.7 ≈ 64.3 µm2

Width and height of core area = sqrt 64.3 = 8 µm

So for keeping 50% core utlization I kept my Height and Weidth of core 12µm.



Core utilization for a simple counter is kept as 0.4 for easy routing and optimization. Core to IO boundaries is kept as 13 microns. This is kept w.r.t power ring to be placed in next step. You will get a better understanding after power planning step.


13.080
10
0
0
total=12.060
0.060
12.000
10
0
10
total=12.050
12.040

Power Planning:

Power planning is an essential part of the ASIC physical design flow, ensuring the chip receives a stable and reliable supply of power to all standard cells, macros, memories, and I/Os.

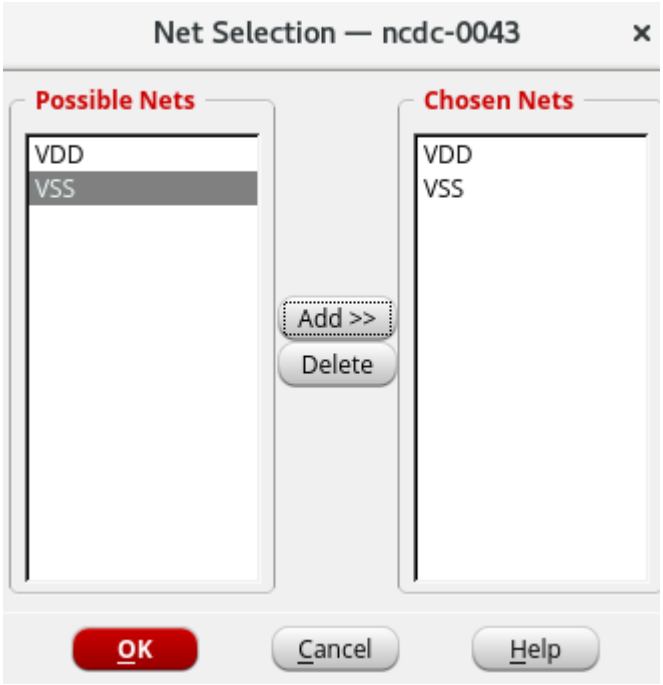
- Ensure voltage levels remain stable throughout the chip.
- Minimize IR drop (voltage drop due to resistance in metal).
- Avoid electromigration (metal damage due to high current).
- Provide robust power to all functional blocks and corners.

PDN is the actual physical implementation of the power plan. A hierarchical network of wires that deliver power from the chip's IO pads (or bumps) to every internal standard cell and macro.

PDN Includes:

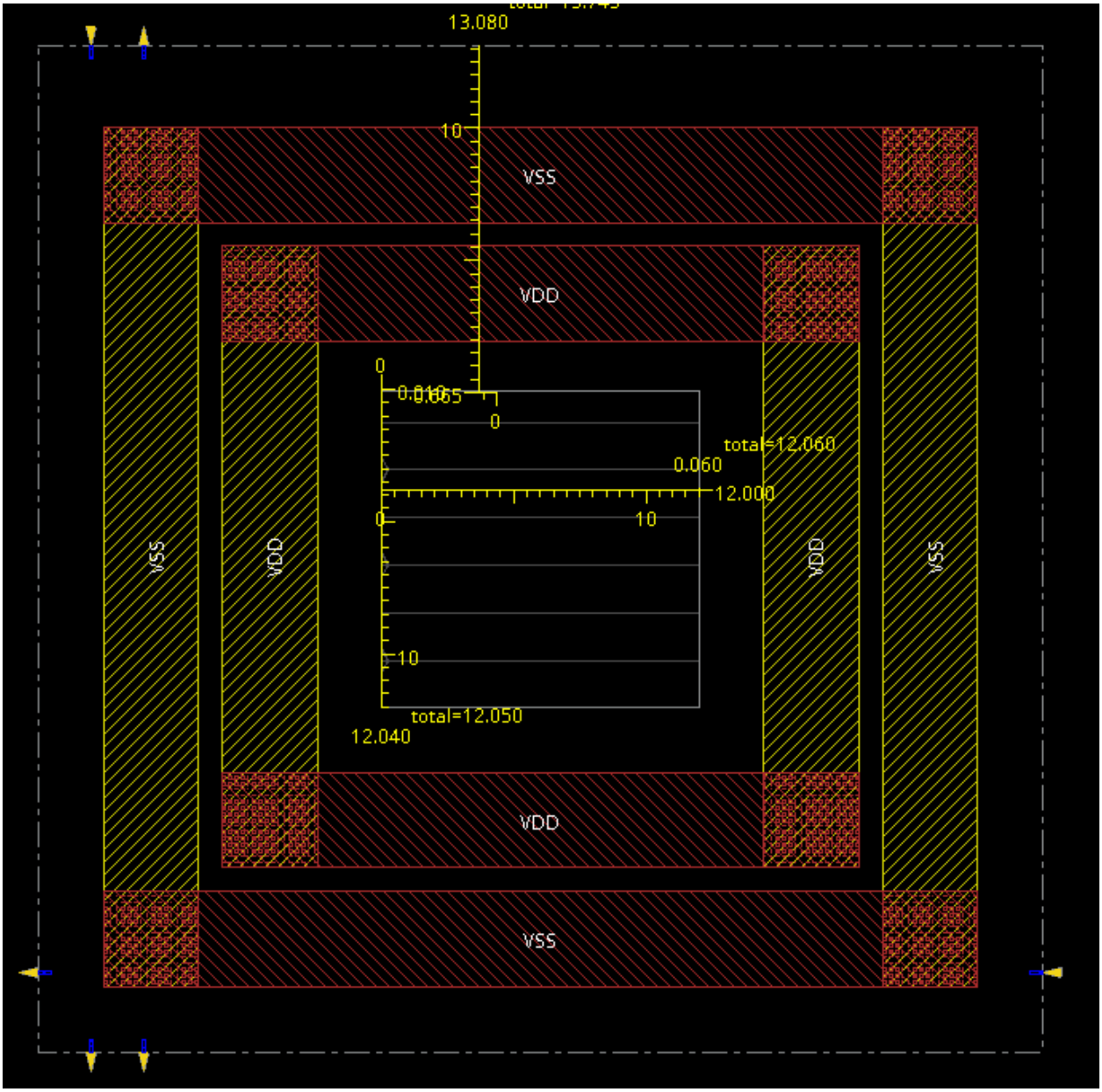
- IO pads → Core rings → Power stripes → Standard cells
- Multiple metal layers used to reduce resistance
- Connections to decoupling capacitors to stabilize power
- Click on Power => Power planning => Add Ring.
- Add Nets (e.g. VDD, VSS)
- For power ring higher metal layers are chosen such as metal layer 5 and 4 for wider tracks, low resistance and higher current capacity with low IR drop.
- Spacing b/w rings, width and offset from boundary is set in compliance with DRC rule check given by foundry.

Use top and bottom layers at M5 and right and left metal layers as M4.



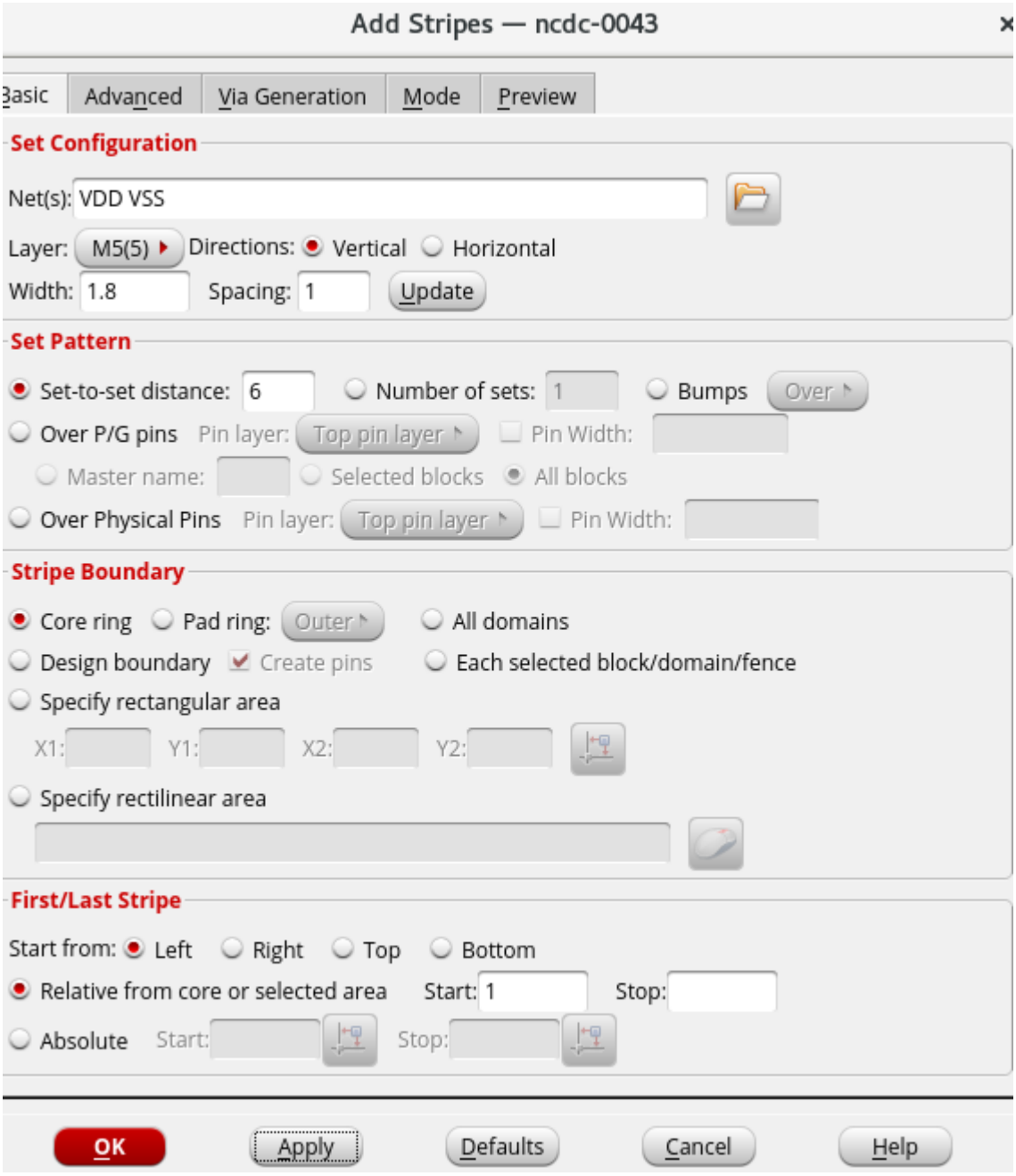
For my design I have choose

- Power ring width = 3.6 µm
- Spacing between VDD and GND = 0.9 µm
- Core to IO boundary = 3.6 (VDD) + 2 + 3.6(GND) = 9.2 µm of space around the core
- Offset for power rails = 13-9.2 = 3.8µm.

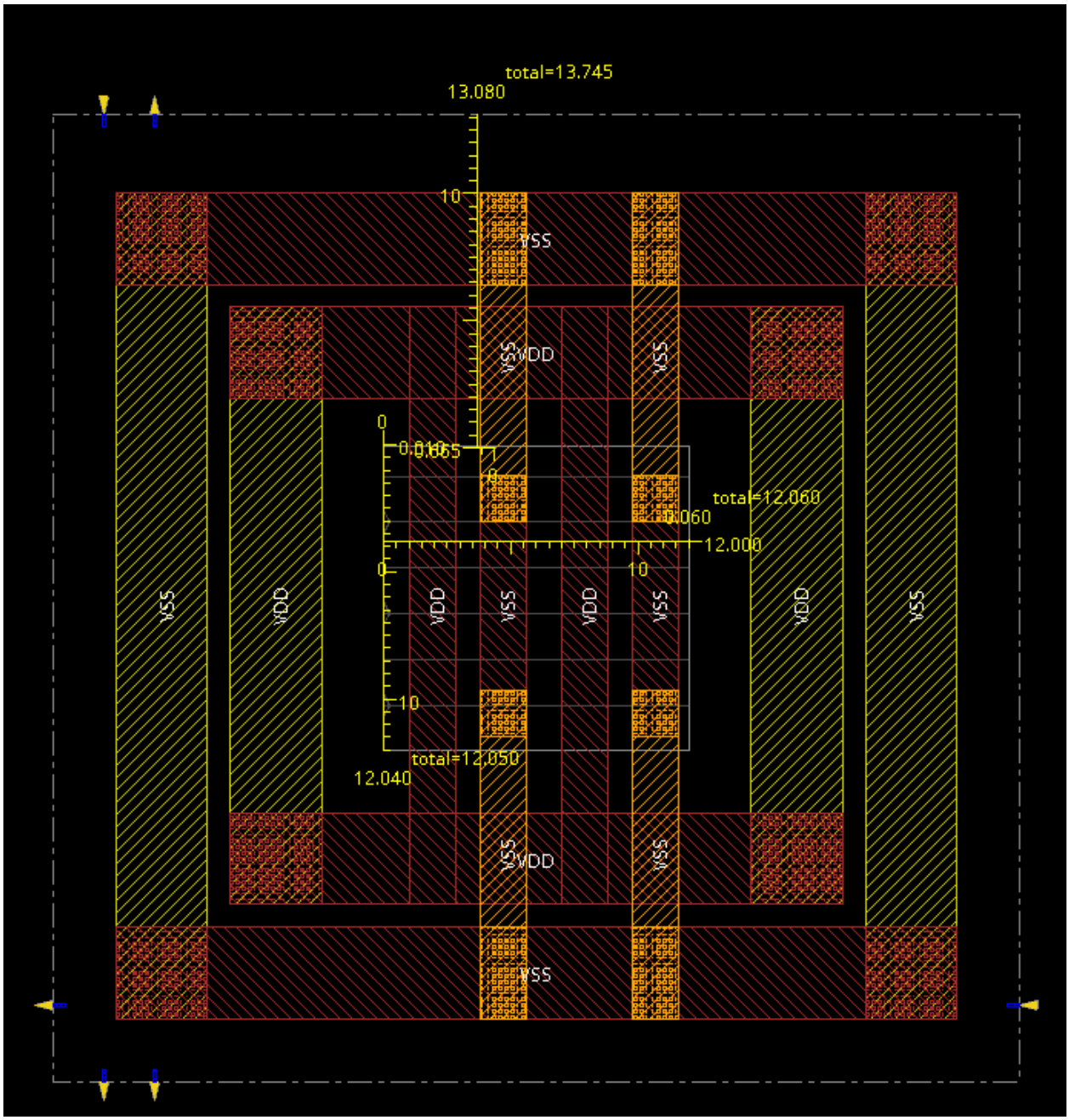


After placing a power ring around the core, metal stripes (VDD and GND) are inserted horizontally and vertically inside the core to:

- Reduce IR drop (voltage drop).
- Provide multiple power taps to standard cells.
- Support current demand eenly across the layout.
- For power strips, click on power => Power planning=> Add strips
- Enter the following information that is shown by red box.

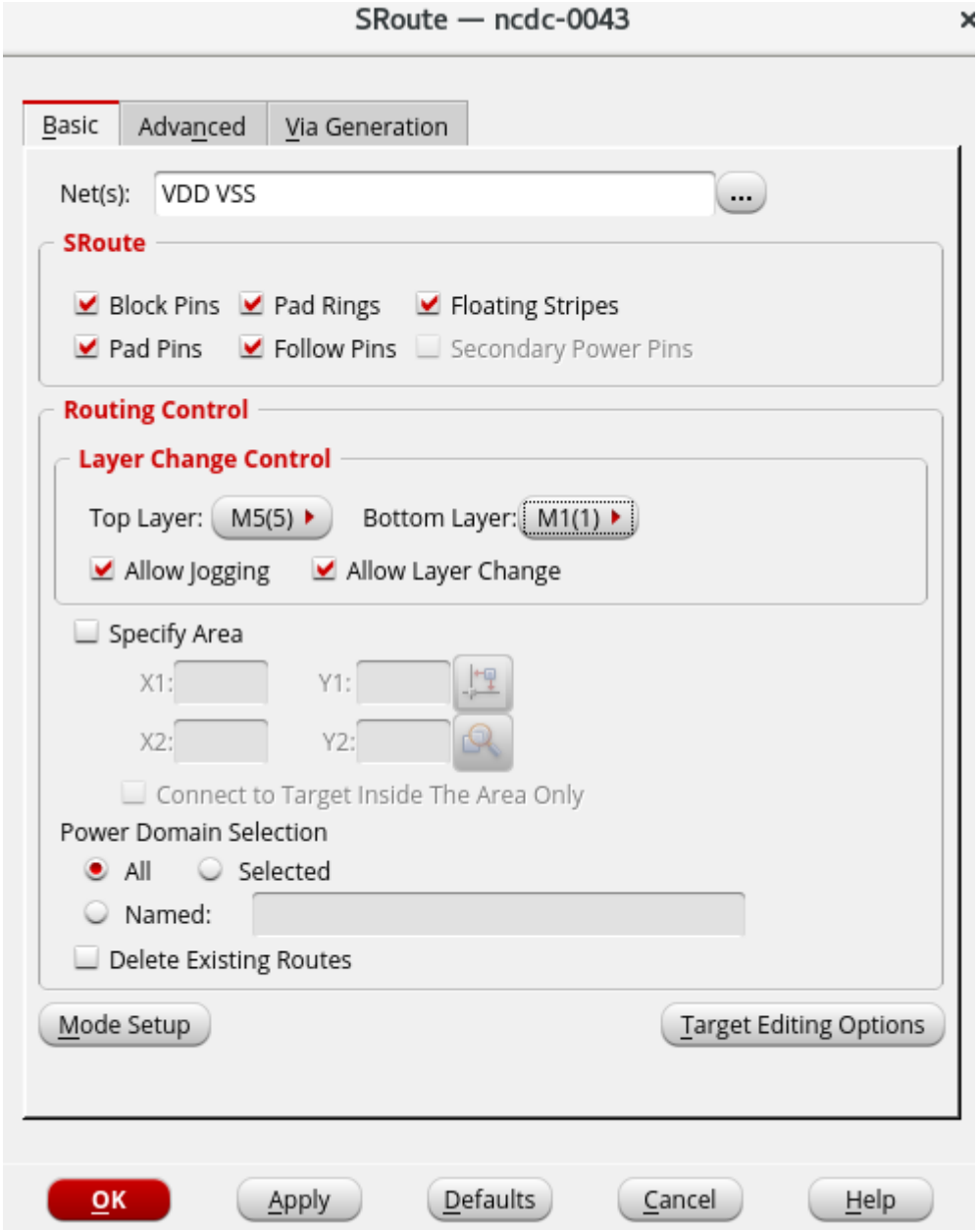


Now, your design looks like this:



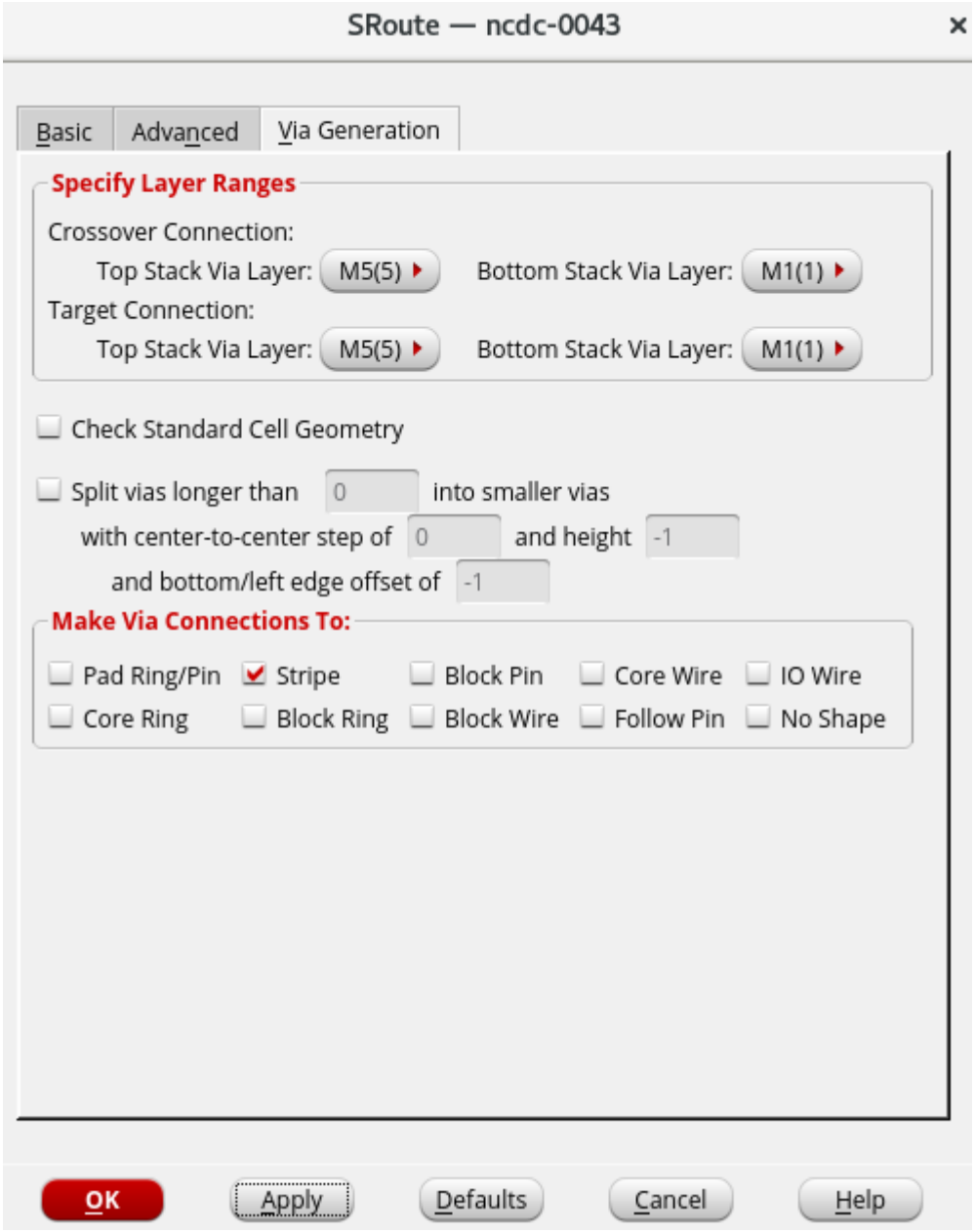
Now, creating power rails with special route. Go to Route -> Special route

I used M5 for power ring, so I will select M5 as top layers while specifying power rails in my design. Vias need to be connected from power rails to power rings to get power so select same metal layer you used for power ring.



Now, creating power rails with special route.

I went to Route -> Special route I used M5 for power ring, so I will select M5 as top layers while specifying power rails in my design. Vias need to be connected from power rails to power rings to get power so select same metal layer you used for power ring.



Zoom in to see that power routes have been connected to the power planned targets with relevant vias.

Global Connections:

VDD or VSS Net

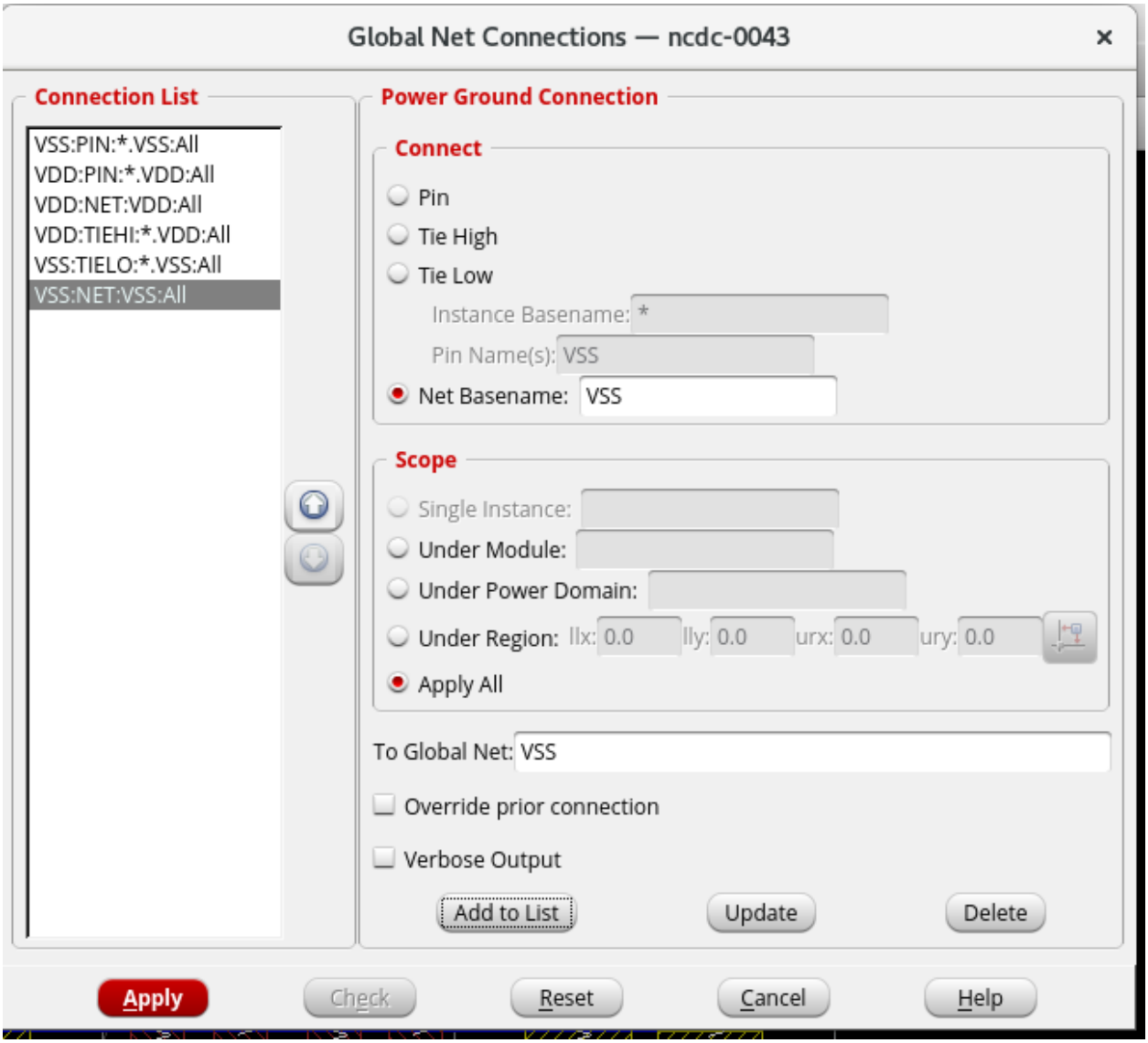
- These are the names of the global power and ground nets used throughout the design.
- They connect to the power/ground rails, stripes, and rings.
- Refers to the power/ground pins of standard cells or macros. VDD TIEHI & VSS TIELO
- These refer to cells that provide a constant logic high (1) or low (0).
- Used to tie unused inputs or force constant values.

Without it, the tool may:

- Not recognize VDD or VSS connections properly

- Leave cells unpowered
- Fail LVS or IR drop checks
- Cause physical connectivity issues (like stripes/rings not actually powering logic)
- For Global connections, click on Power => Connect Global Nets.
- The new window will be open, enter the following information that is shown in the below figure: VDD as pin, net and then TIEHI, same with VSS. Select one by one and do add to list.

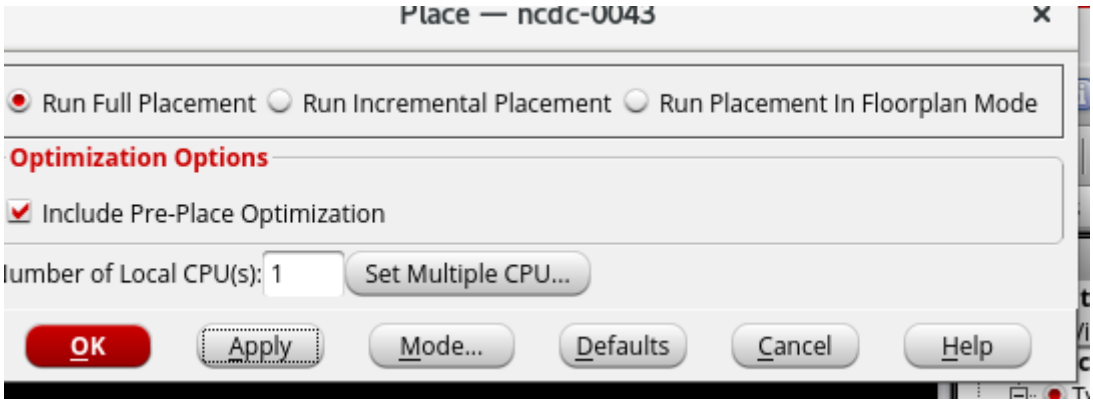
Keep reading your terminal after performing each step. Check for any errors in terminal. Stop there and correct the errors.

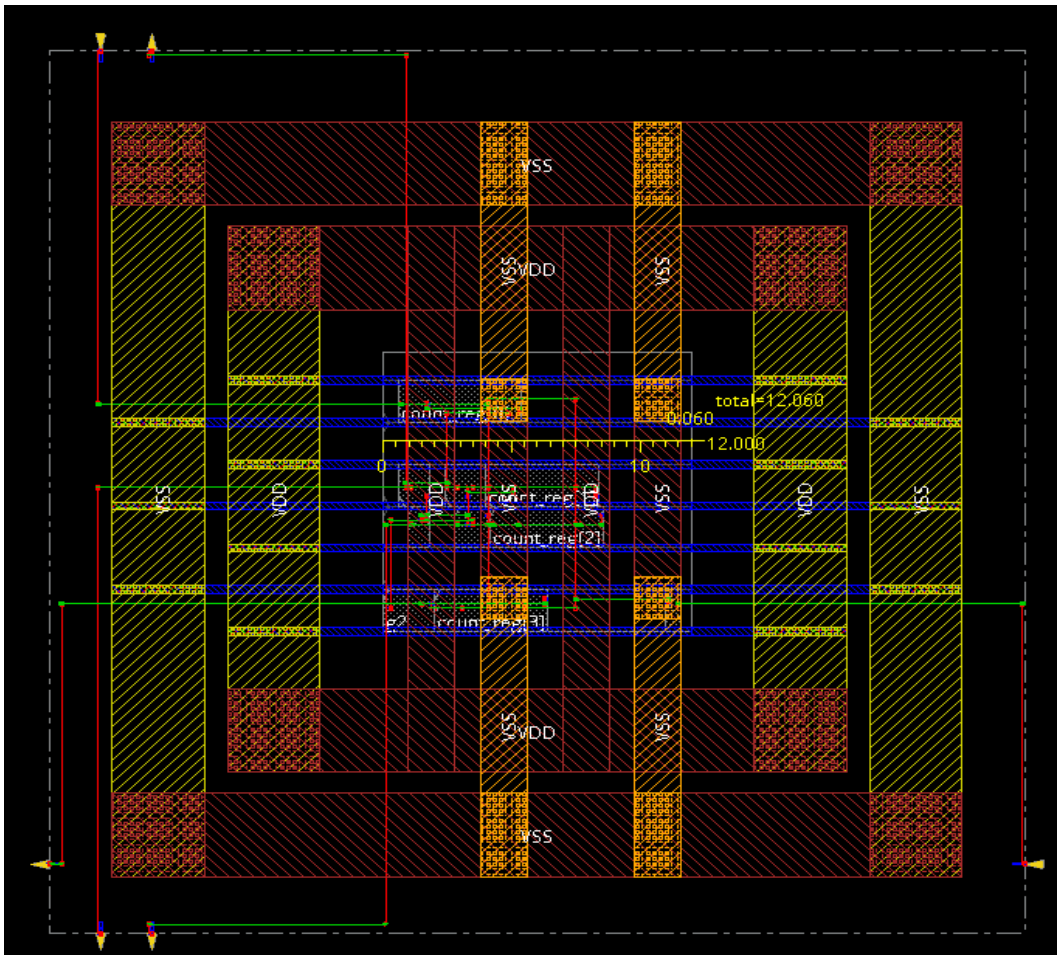


Placement:

Placement is the stage in the physical design flow where the standard cells (logic gates) are assigned physical locations on the silicon chip, without overlapping, based on the logical netlist generated from synthesis.

- We will place standard cells in our core area. These std cells are basically our RTL logic mapped to std cells from pdk.





Clock Tree Synthesis:

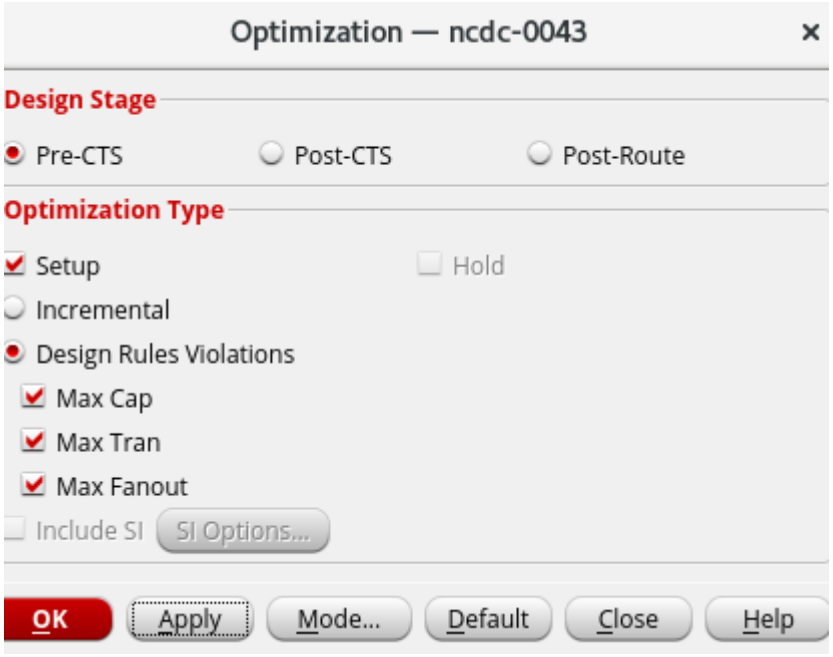
Clock Tree Synthesis (CTS) is a crucial stage in the ASIC backend flow that ensures the clock signal reaches all sequential elements (flip-flops, latches) with minimal skew and optimal delay.

First, we will perform **Pre-CTS:**

Its main purpose is to prepare the design for successful CTS and post-CTS timing closure.

- Fix setup timing violations on data paths.
- Prepare for clock tree balancing
- Reduce congestion and improve cell placement
- Minimize clock skew impact
- Add buffers to improve transition, fanout and capacitance.
- Legalize placement of cells to ensure no overlaps and DRC errors.

For this go to ECO -> Optimize design. Check Max Fanout.



Click on apply. Check your terminal and read it.

```
[NR-eGR] total congestion hotspot area = 0.00 (area is in
[NR-eGR]              Length (um)  Vias
[NR-eGR] ------------------------------
[NR-eGR]  M1  (1H)              0    42
[NR-eGR]  M2  (2V)            124    52
[NR-eGR]  M3  (3H)            125     0
[NR-eGR]  M4  (4V)              0     0
[NR-eGR]  M5  (5H)              0     0
[NR-eGR]  M6  (6V)              0     0
[NR-eGR]  M7  (7H)              0     0
[NR-eGR]  M8  (8V)              0     0
[NR-eGR]  M9  (9H)              0     0
[NR-eGR]  AP  (10V)             0     0
[NR-eGR] ------------------------------
[NR-eGR]      Total           250    94
[NR-eGR] ------------------------------------------
```

Comment:

This report shows the routing summary of a chip design, listing metal layers (M1–M9 and AP) along with the wire length and number of vias used in each. Most of the routing happens in M2 (131 µm, 60 vias) and M3 (129 µm, no vias), while other layers are unused. M1 mainly has vias (44) but no wire length. Overall, the total routing length is 260 µm with 104 vias, indicating a small design with minimal congestion and simple metal layer usage.

No negative slack:

```
Setup views included:
 slow

+--------------------+---------+---------+---------+
|     Setup mode     |   all   | reg2reg | default |
+--------------------+---------+---------+---------+
|           WNS (ns):|  0.850  |  3.481  |  0.850  |
|           TNS (ns):|  0.000  |  0.000  |  0.000  |
|    Violating Paths:|    0    |    0    |    0    |
|          All Paths:|   12    |    4    |    8    |
+--------------------+---------+---------+---------+
```

This report shows the timing summary for the design under the slow corner (setup analysis). The Worst Negative Slack (WNS) is 0.849 ns, and the Total Negative Slack (TNS) is 0.000 ns, meaning there are no timing violations. It also shows that there are 12 total paths analyzed and 0 violating paths, indicating that the design comfortably meets its setup timing requirements and is performing well under the given conditions.

- Make sure there will be no error.
- Do clock-tree synthesis by writing the commands in terminal
- create_ccopt_clock_tree_spec -file ccopt.spec

```
innovus 2> create_ccopt_clock_tree_spec -file ccopt.spec
Creating clock tree spec for modes (timing configs): constraints
extract_clock_generator_skew_groups=true: create_ccopt_clock_tree_spec will gene
rate skew groups with a name prefix of "_clock_gen" to balance clock generator c
onnected flops with the clock generator they drive.
Reset timing graph...
Ignoring AAE DB Resetting ...
Reset timing graph done.
Ignoring AAE DB Resetting ...
Analyzing clock structure...
Analyzing clock structure done.
Reset timing graph...
Ignoring AAE DB Resetting ...
Reset timing graph done.
Wrote: ccopt.spec
```

source ccopt.spec

```
innovus 3> source ccopt.spec
Extracting original clock gating for clk...
  clock_tree clk contains 4 sinks and 0 clock gates.
Extracting original clock gating for clk done.
The skew group clk/constraints was created. It contains 4 sinks and 1 sources.
Checking clock tree convergence...
Checking clock tree convergence done.
```

ccopt_design

Again, read your terminal carefully for timing and no error should be displayed.

```
*** Summary of all messages that are not suppressed in this session:
Severity  ID               Count  Summary
WARNING   IMPEXT-3530          5  The process node is not set. Use the com...
WARNING   IMPSP-9025           1  No scan chain specified/traced.
WARNING   IMPOPT-3564          1  The following cells are set dont_use tem...
WARNING   IMPTCM-77            2  Option "%s" for command %s is obsolete a...
*** Message Summary: 9 warning(s), 0 error(s)
```

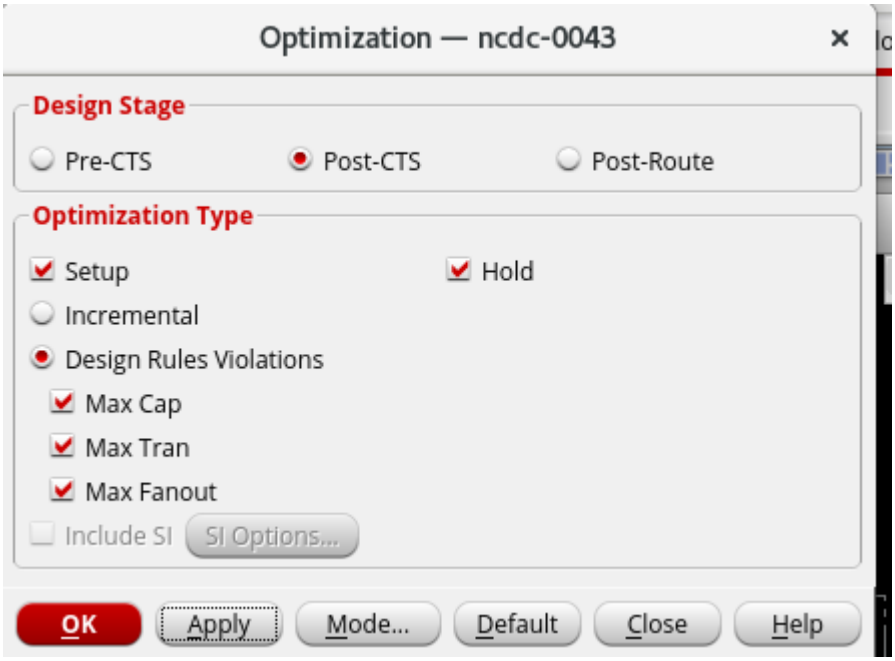
Explanation:

The command create_ccopt_clock_tree_spec -file ccopt.spec generates the clock tree specification file that defines how the clock distribution network will be built in the design. The terminal output shows that Innovus analyzed the clock structure, identified clock sources, and created skew groups to balance the clock paths. It also reset the timing graph multiple times for accurate analysis and finally wrote the ccopt.spec file successfully, which will be used in the next clock tree synthesis step.It optimizes and builds the clock network based on the specifications defined in the ccopt.spec file, aiming to minimize skew, latency, and power consumption while ensuring that the clock signal reaches all flip-flops simultaneously and efficiently.

Post-CTS optimization with respect to Setup and Hold

Click on apply.

Terminal after CTS:



Read your terminal carefully and understand it. Comment on the terminal displays.

```
Setup views included:
 slow
Hold views included:
 fast

+--------------------+---------+---------+---------+
|     Setup mode     |   all   | reg2reg | default |
+--------------------+---------+---------+---------+
|           WNS (ns):|  0.891  |  3.484  |  0.891  |
|           TNS (ns):|  0.000  |  0.000  |  0.000  |
|    Violating Paths:|    0    |    0    |    0    |
|          All Paths:|   12    |    4    |    8    |
+--------------------+---------+---------+---------+

+--------------------+---------+---------+---------+
|     Hold mode      |   all   | reg2reg | default |
+--------------------+---------+---------+---------+
|           WNS (ns):|  0.074  |  0.074  |  0.000  |
|           TNS (ns):|  0.000  |  0.000  |  0.000  |
|    Violating Paths:|    0    |    0    |    0    |
|          All Paths:|    4    |    4    |    0    |
+--------------------+---------+---------+---------+
```

Comment:

This timing report shows that the design has no setup or hold violations, meaning the clock tree synthesis was successful. The Worst Negative Slack (WNS) values are positive for both setup (0.891 ns) and hold (0.074 ns), indicating that all timing paths meet the required constraints. The Total Negative Slack (TNS) is 0,

confirming there are no failing paths. Overall, the clock network is well-balanced, and timing closure has been achieved successfully.

Routing:

Routing is a key step in digital physical design where the electrical connections (wires) between the placed standard cells and macros are physically created using available metal layers on the chip.

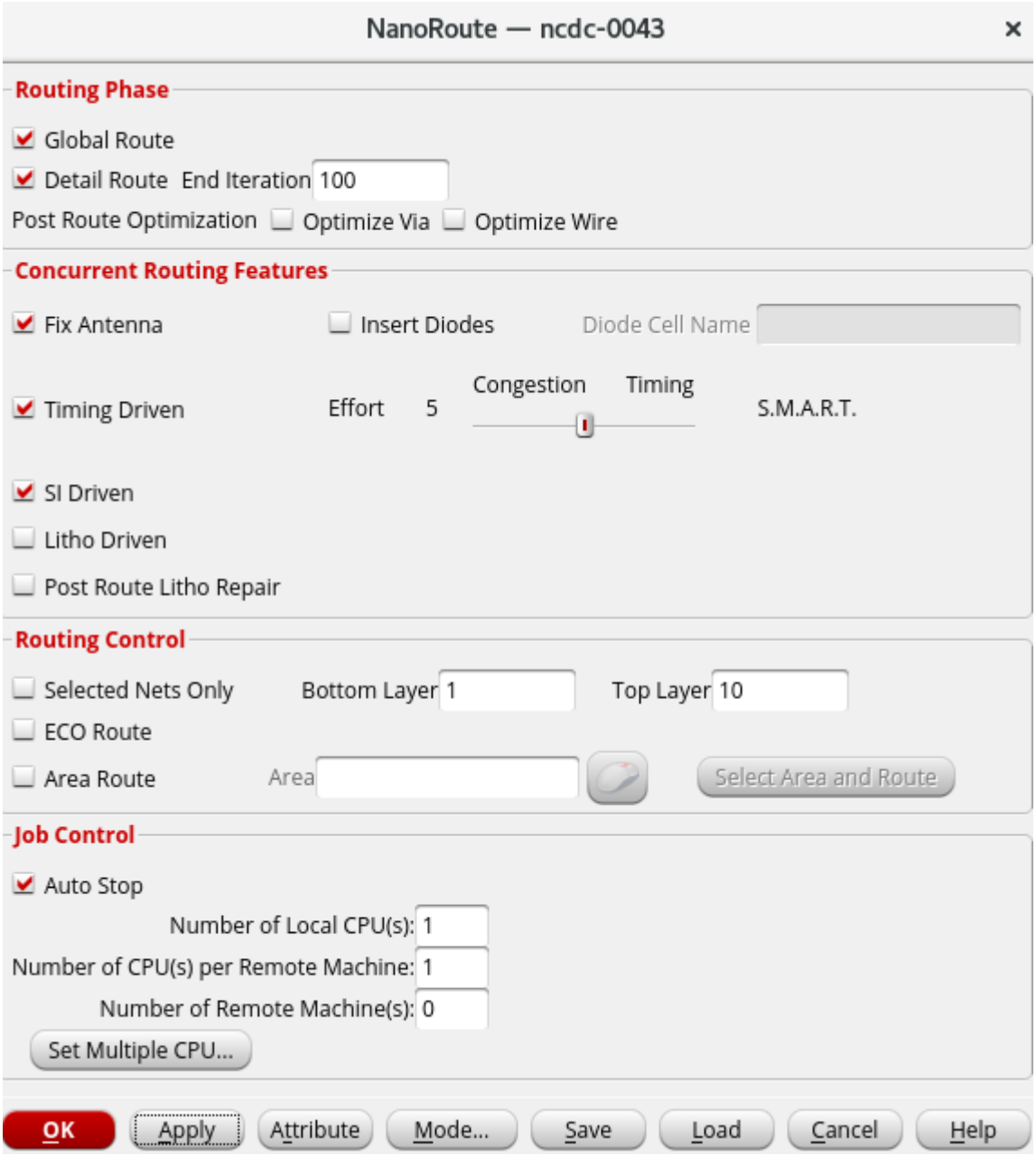
1. **Global Routing**

Estimates the best routing paths based on congestion and available routing resources.

2. **Detailed Routing**

Performs the actual physical connection of wires between pins and cells.

- Ensure all nets are connected without open or short circuits.
- Avoid DRC violations (e.g., spacing, width).
- Minimize congestion, delay, skew, and power.
- Maintain timing closure: wires shouldn't cause setup/hold violations.
- Click on Route => NanoRoute => Route (Change End Iteration to 100)



Comment:

This report shows the wire length and via distribution after routing. The total wire length is 260 µm, with most routing done on M1 (72 µm) and M2 (114 µm) layers, while higher metal layers were not used. The total number of vias is 77, mainly between lower layers — 49 vias on M1, 21 on M2, and 7 on M3 — indicating short interconnections and efficient routing. Overall, the design uses minimal routing resources with no excessive metal usage or unnecessary layer transitions, which reflects a compact and optimized layout.

```
#Post Route wire spread is done.
#Total number of nets with non-default rule or having extra spacing = 1
#Total wire length = 256 um.
#Total half perimeter of net bounding box = 251 um.
#Total wire length on LAYER M1 = 79 um.
#Total wire length on LAYER M2 = 114 um.
#Total wire length on LAYER M3 = 47 um.
#Total wire length on LAYER M4 = 15 um.
#Total wire length on LAYER M5 = 0 um.
#Total wire length on LAYER M6 = 0 um.
#Total wire length on LAYER M7 = 0 um.
#Total wire length on LAYER M8 = 0 um.
#Total wire length on LAYER M9 = 0 um.
#Total wire length on LAYER AP = 0 um.
#Total number of vias = 74
#Up-Via Summary (total 74):
#
#-----------------------
# M1                 52
# M2                 20
# M3                  2
#-----------------------
#                    74
#
```

```
*** Summary of all messages that are not suppressed in this session:
Severity  ID               Count  Summary
WARNING   NRGR-22              3  Design is already detail routed.
*** Message Summary: 2 warning(s), 0 error(s)
```

SPEF FILE:

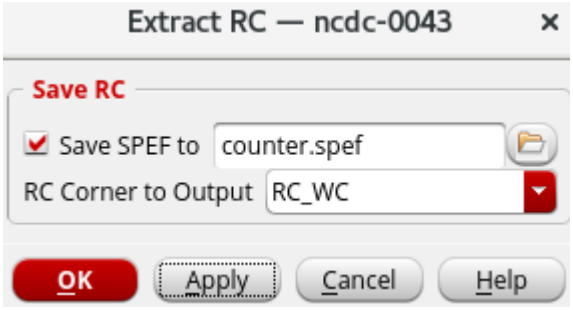
After routing, spef file is generated. It gives detailed electrical models of interconnects i.e R and C to model delays. This file is used in tempus for STA, also used in power analysis and SI analysis.

```
*SPEF "IEEE 1481-1998"
*DESIGN "counter"
*DATE "Tue Oct  7 18:43:56 2025"
*VENDOR "Cadence Design Systems, Inc."
*PROGRAM "Innovus"
*VERSION "21.18-s099_1"
*DESIGN_FLOW "PIN_CAP NONE" "NAME_SCOPE LOCAL"
*DIVIDER /
*DELIMITER :
*BUS_DELIMITER []
*T_UNIT 1 NS
*C_UNIT 1 PF
*R_UNIT 1 OHM
*L_UNIT 1 HENRY

// MMMC spef file for corner 'RC_WC'

*NAME_MAP

*1 clk
*2 reset
*3 count[3]
*4 count[2]
*5 count[1]
*6 count[0]
*7 n_0
*8 n_2
*9 n_4
*10 n_5
*11 n_7
*12 n_9
*13 n_12
*14 count_reg\[3\]
*15 count_reg\[2\]
*16 g83__5107
*17 count_reg\[1\]
*18 g85__6260
*19 g86__4319
*20 g88__8428
*21 g92
*22 g2
*23 count_reg\[0\]
```



Purpose of SPEF File:

The SPEF (Standard Parasitic Exchange Format) file contains detailed post-layout parasitic data of the design "counter," generated by Cadence Innovus. It specifies units for time, capacitance, resistance, and inductance, and includes sections like NAME_MAP for signal mapping, PORTS for I/O details, and D_NET for net-level parasitic information. Each net, such as the clock, lists its total capacitance and individual segment values, enabling accurate timing and signal integrity analysis after routing.

RC Extraction & Timing Analysis:

Run setup and hold timing analysis.

Use the following commands and read your terminal for setup/hold vilations.

Set_db timing_analysis_type ocv

```
innovus 9> set_db timing_analysis_type ocv
1 ocv
innovus 10> report_timing -path_type full_clock -max_paths 10
#################################################################
#  Generated by:      Cadence Innovus 21.18-s099_1
#  OS:                Linux x86_64(Host ID ncdc-0043)
```

For setup:

report_timing -path_type full_clock -max_paths 10

```
Path 10: MET Setup Check with Pin count_reg[2]/CP
Endpoint:   count_reg[2]/D (^) checked with  leading edge of 'clk'
Beginpoint: count_reg[1]/Q (v) triggered by  leading edge of 'clk'
Path Groups: {clk}
Analysis View: slow
Other End Arrival Time          0.000
- Setup                         0.055
+ Phase Shift                   4.180
- Uncertainty                   0.010
= Required Time                 4.115
- Arrival Time                  0.555
= Slack Time                    3.560
     Clock Rise Edge                     0.000
     + Source Insertion Delay           -0.000
     = Beginpoint Arrival Time          -0.000
     Timing Path:
```

Comment on terminal output:

This report shows the setup timing check for the register count_reg[2] in the slow corner timing view. The required time for the signal to arrive is 9.935 ns, while the actual arrival time is 0.564 ns, giving a large positive slack of 9.371 ns. This means the signal reaches well before the required time, so there's no setup violation. The phase shift of 10 ns indicates the clock period, and small uncertainty (0.010 ns) represents minimal timing variation. Overall, the path is very safe and easily meets setup timing requirements

For Hold:

setAnalysisMode -checkType hold

report_timing -check_type hold -path_type full_clock -max_paths 10

```
innovus 11> setAnalysisMode -checkType hold
innovus 12> report_timing -check_type hold -path_type full_clock -max_paths 10
#############################################################
#  Generated by:      Cadence Innovus 21.18-s099_1
#  OS:                Linux x86_64(Host ID ncdc-0043)
#  Generated on:      Tue Oct  7 18:46:45 2025
#  Design:            counter
#  Command:           report_timing -check_type hold -path_type full_clock -max_paths 10
#############################################################
AAE_INFO: opIsDesignInPostRouteState() is 1
#################################################################################
# Design Stage: PostRoute
# Design Name: counter
# Design Mode: 90nm
# Analysis Mode: MMMC OCV
# Parasitics Mode: No SPEF/RCDB
# Signoff Settings: SI Off
#################################################################################
Start delay calculation (fullDC) (1 T). (MEM=2579.72)
*** Calculating scaling factor for BC_LIBSET libraries using the default operating condition of each library.
Total number of fetched objects 13
AAE_INFO: Total number of nets for which stage creation was skipped for all views 0
End delay calculation. (MEM=2599.45 CPU=0:00:00.0 REAL=0:00:00.0)
End delay calculation (fullDC). (MEM=2599.45 CPU=0:00:00.0 REAL=0:00:00.0)
Path 1: MET Hold Check with Pin count_reg[0]/CP
```

The terminal output presents the results of a hold time analysis on the specified path, identified as "count_reg[1]/CP." The analysis indicates that the required time for the signal to stabilize is 0.032 ns, with an actual arrival time of 0.158 ns. This results in a timing slack of 0.126 ns, suggesting that the hold time requirement is met comfortably. Additionally, the output shows minimal uncertainty and no significant phase shift, indicating a well-timed path with respect to the leading edge of the clock signal. Overall, the analysis confirms that the design is robust in terms of hold time specifications.

To switch back to setup later:

setAnalysisMode -checkType setup

```
Path 3: MET Hold Check with Pin count_reg[3]/CP
Endpoint:   count_reg[3]/D (v) checked with  leading edge of 'clk'
Beginpoint: count_reg[3]/Q (v) triggered by  leading edge of 'clk'
Path Groups: {clk}
Analysis View: fast
Other End Arrival Time          0.000
+ Hold                          0.022
+ Phase Shift                   0.000
+ Uncertainty                   0.010
= Required Time                 0.032
  Arrival Time                  0.149
  Slack Time                    0.117
     Clock Rise Edge                      0.000
     = Beginpoint Arrival Time            0.000
     Timing Path:
```

Physical Verification:

Design Rule Check (DRC):

DRC ensures that your layout follows all the manufacturing rules imposed by the foundry for the given technology node (e.g., 65nm, 28nm).

- Minimum spacing between wires
- Minimum width of metal tracks
- Via enclosure and spacing
- End-of-line spacing
- Metal density requirements
- Violating DRC can lead to fabrication failures, shorts, opens, or low yield.
- A layout that fails DRC cannot be taped out for manufacturing.

```
innovus 14> #-check_ndr_spacing auto                # enums={true false auto}, default=auto,  user setting
#-check_same_via_cell true              # bool, default=false, user setting
#-report counter.drc.rpt                # string, default="", user setting
 *** Starting Verify DRC (MEM: 2581.7) ***

  VERIFY DRC ..... Starting Verification
  VERIFY DRC ..... Initializing
  VERIFY DRC ..... Deleting Existing Violations
  VERIFY DRC ..... Creating Sub-Areas
  VERIFY DRC ..... Using new threading
  VERIFY DRC ..... Sub-Area: {0.000 0.000 38.000 38.000} 1 of 1
  VERIFY DRC ..... Sub-Area : 1 complete 0 Viols.

  Verification Complete : 0 Viols.
```

No Violations reported:

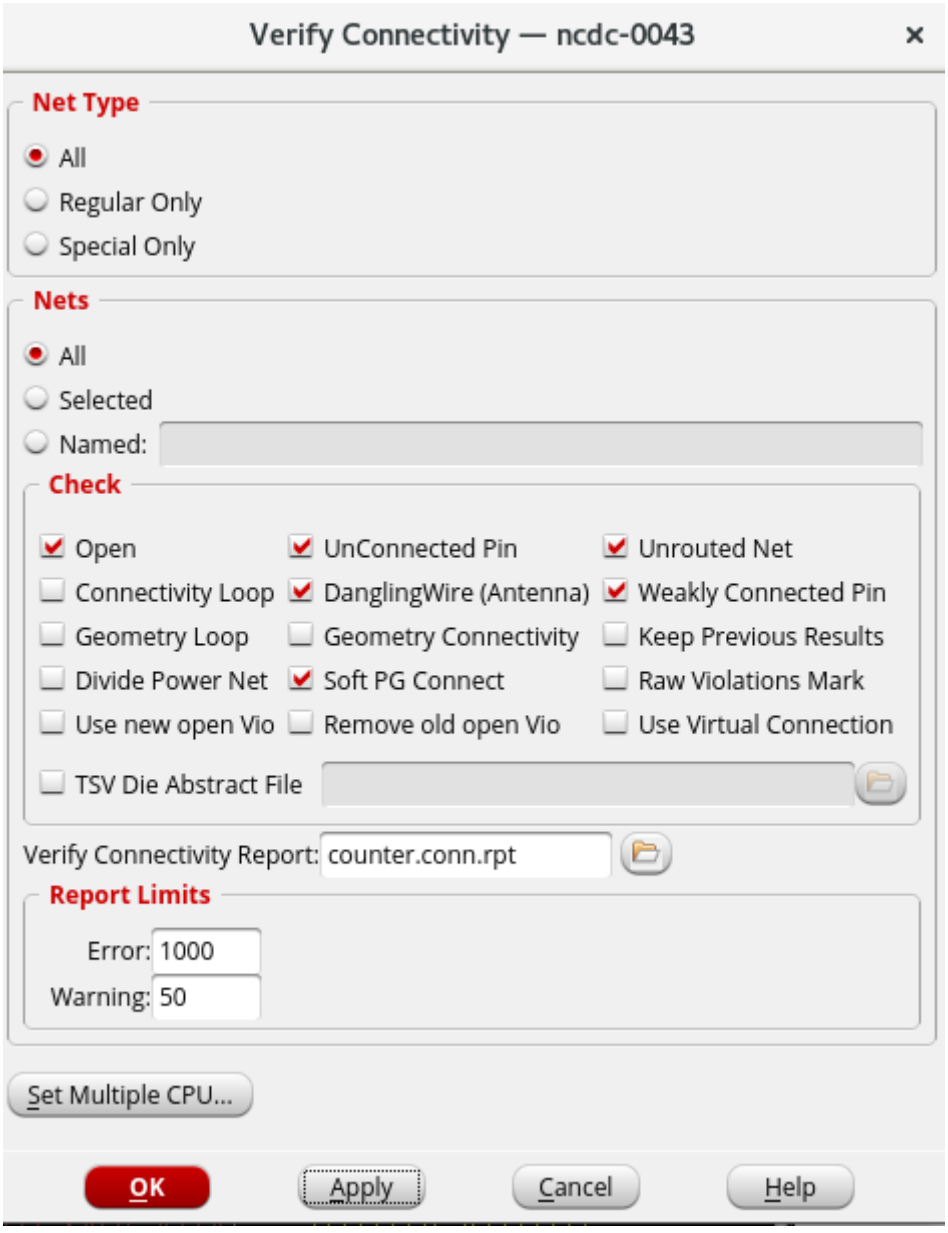
Checking Connectivity:

Connectivity check verifies that all pins, nets, and power/ground connections are properly connected, exactly as described in the logical netlist.

- All nets are fully connected between driver and loads.
- No floating pins or dangling nets.

- No shorts between unintended nets.
- All power (VDD) and ground (VSS) pins are tied to the appropriate rails.
- Connectivity errors can cause functional failures, even if timing and DRC are clean.
- Ensures the physical layout matches the logical intent of the design.

Use default settings.



No Violations reported:

```
Begin Summary
  Found no problems or warnings.
End Summary

End Time: Tue Oct  7 18:50:16 2025
Time Elapsed: 0:00:00.0

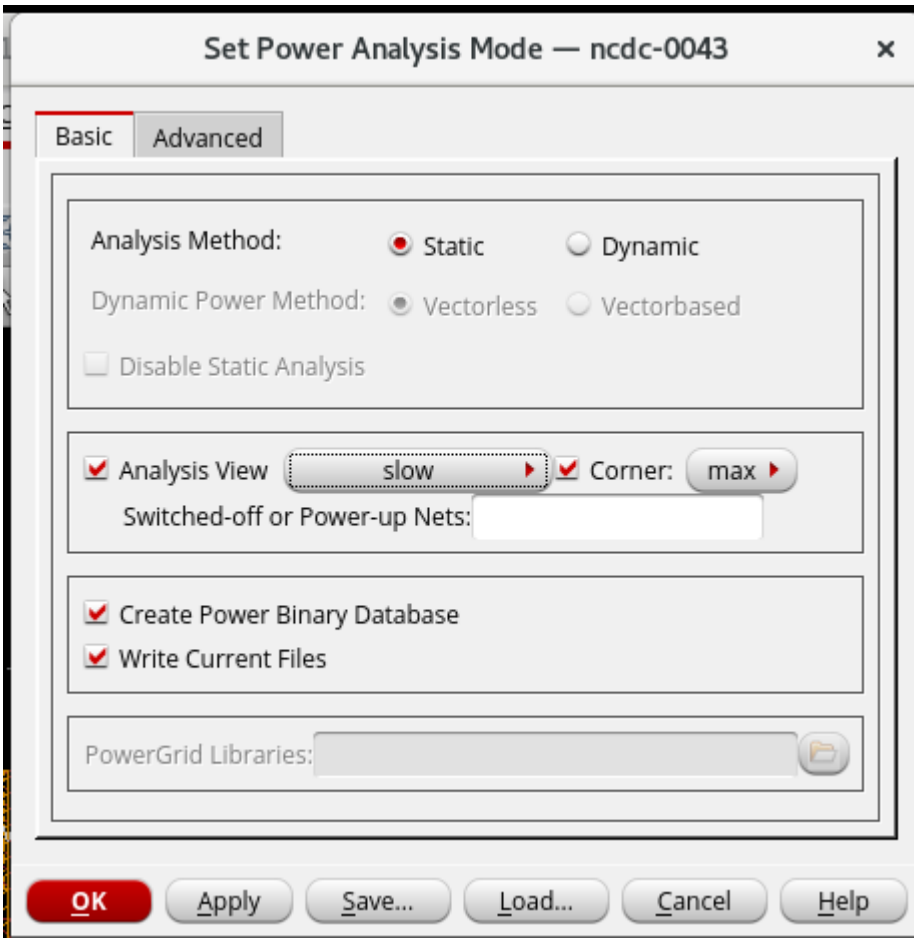
******** End: VERIFY CONNECTIVITY ********
  Verification Complete : 0 Viols.  0 Wrngs.
  (CPU Time: 0:00:00.0  MEM: 0.000M)
```

Power Analysis:

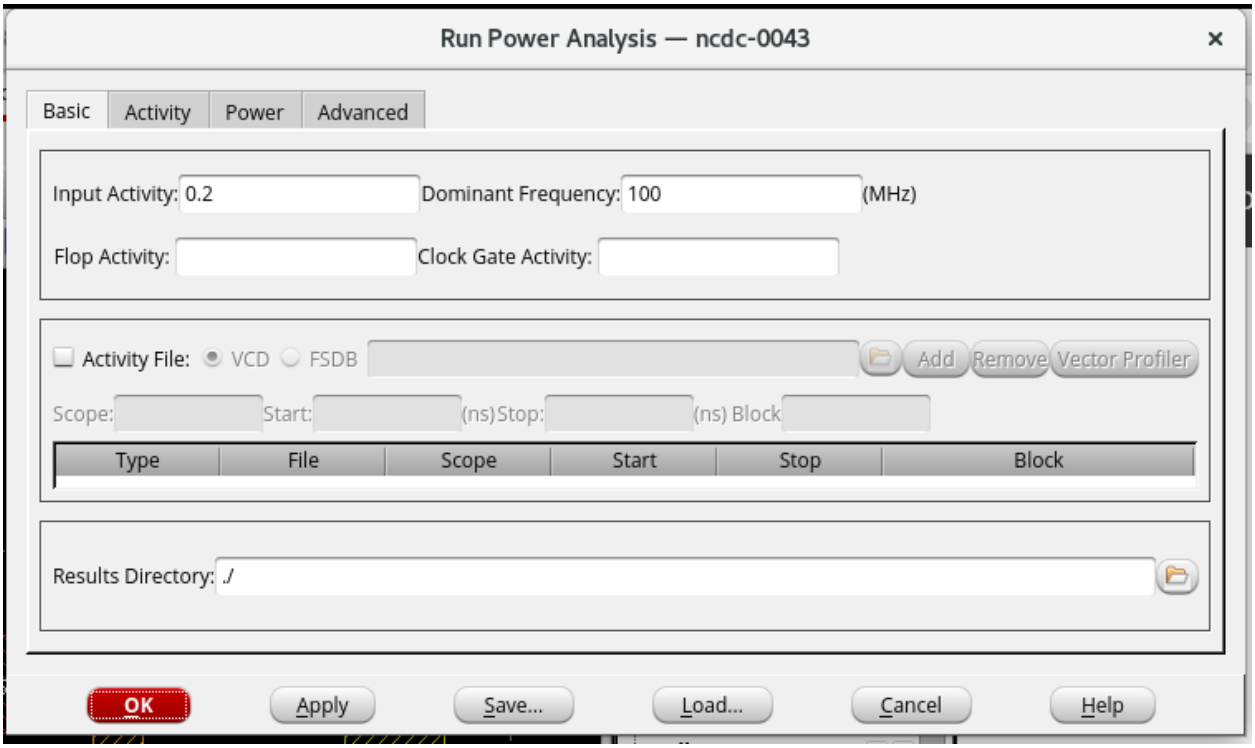
Power analysis is the process of estimating and analyzing the power consumption of chip design.
To ensure it meets power budgets, doesn't overheat, and is efficient for its target application (especially in mobile and battery-operated systems).

- Ensures power budget compliance
- For thermal management
- To minimize leakage and dynamic power
- To prevent IR drop and EM issues
- **Dynamic power** = $\alpha \cdot C \cdot V2 \cdot f$
- **Static Power:** Power consumed when the circuit is idle, due to subthreshold leakage, gate leakage, etc.
- **Short circuit power**: Due to momentary short paths when both PMOS and NMOS are ON during switching.

power -> power analysis ->setup



- Go to power -> power analysis -> Run Use default settings.



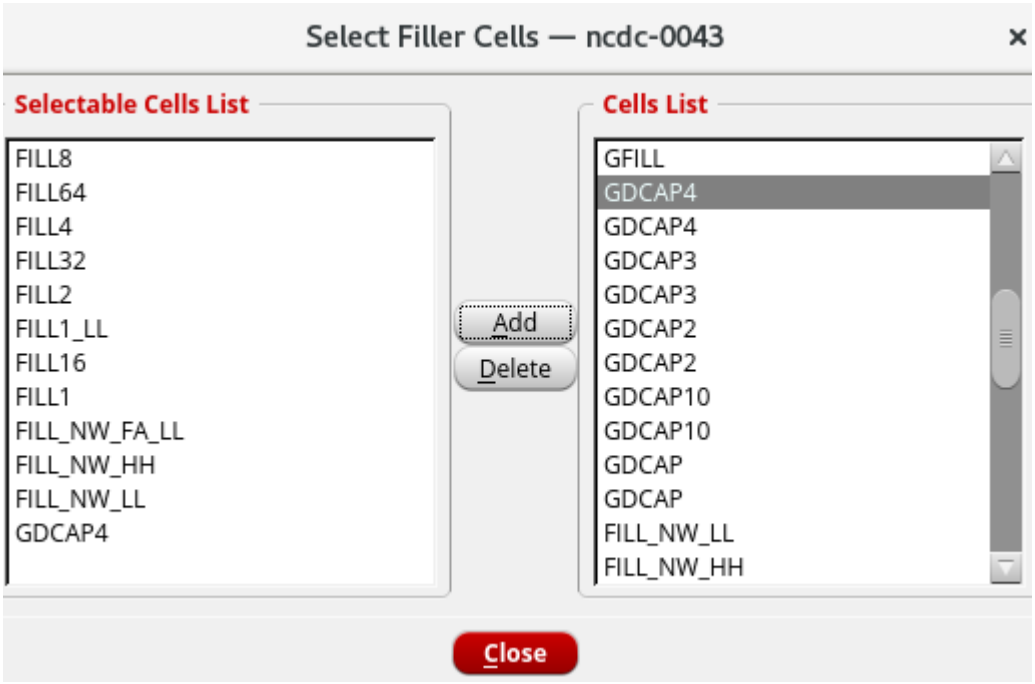
Terminal showing power report:

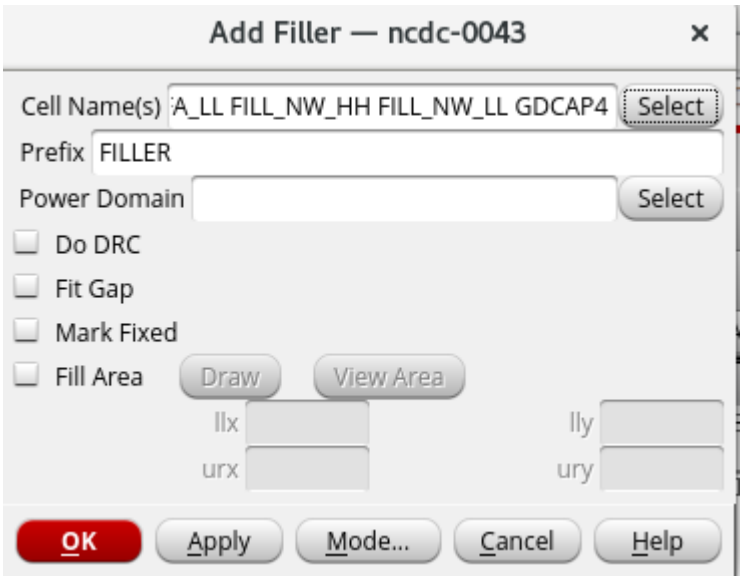
```
Total Power
-----------------------------------------------------------------------------------------
Total Internal Power:        0.01023698            90.1546%
Total Switching Power:       0.00110601             9.7403%
Total Leakage Power:         0.00001193             0.1051%
Total Power:                 0.01135492
-----------------------------------------------------------------------------------------
```

Filler Cells Placement:

Filler cell placement is the process of inserting special, non-functional standard cells (called *filler cells*) into the gaps between placed standard cells after place and route (P&R) is complete.

- Filler cells help maintain continuity and substrate tap spacing.
- They ensure electrical connectivity to VDD/VSS for body biasing and prevent latch-up.
- Foundries have rules requiring minimum metal density. Gaps in layout can violate these. filler cells help maintain uniform metal coverage.
- Standard cells share diffusion and poly with neighbors. If gaps are left unfilled, open diffusion regions may result in leakage or manufacturing issues.
- Without filler cells, you might get DRC errors related to well proximity, diffusion, or density violations.
- Go to Place -> Physical cell -> Add filler

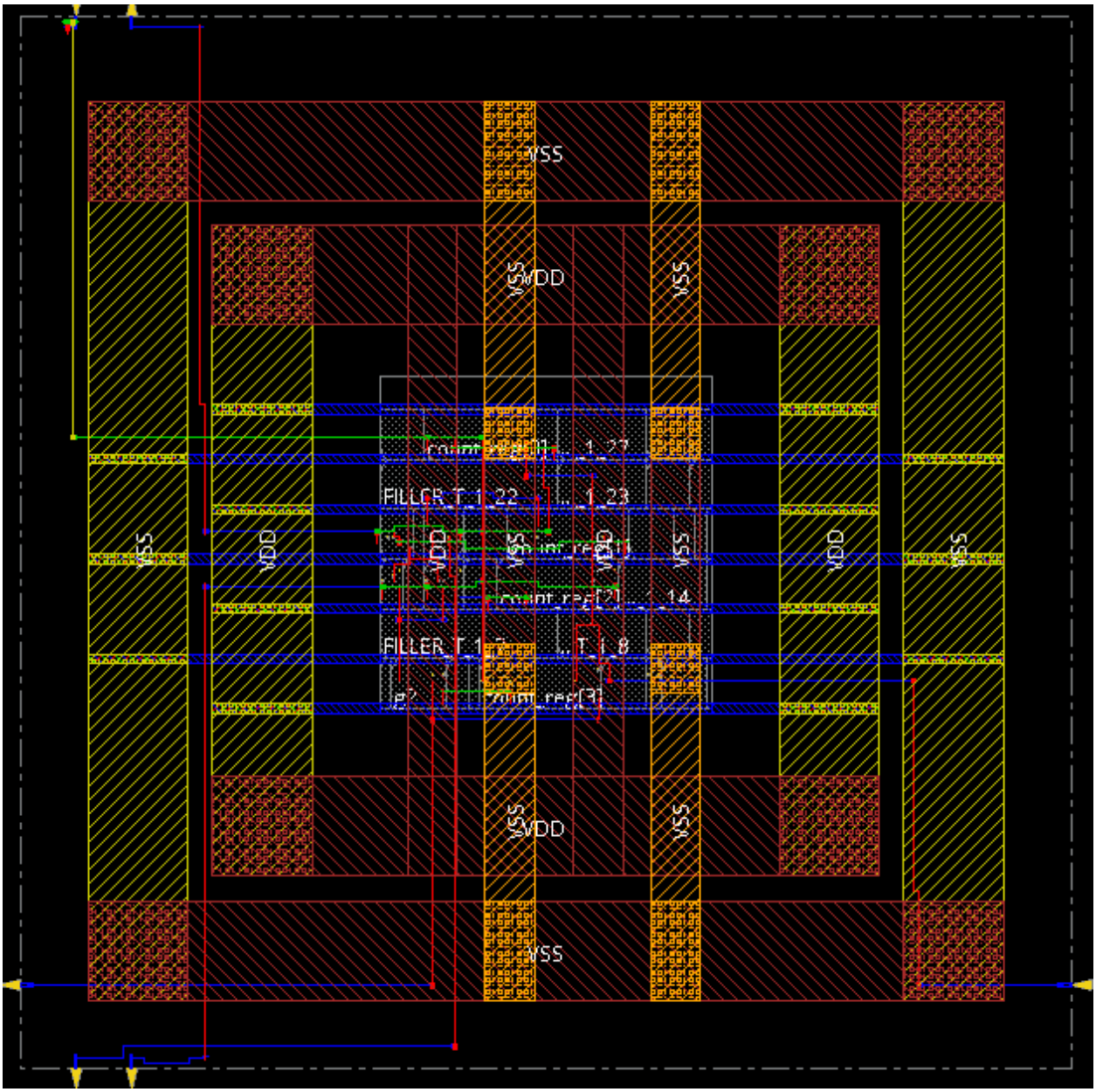




Read terminal.

```
*INFO: Adding fillers to top-module.
*INFO:   Added 0 filler inst of any cell-type.
*INFO: Adding fillers to top-module.
*INFO:   Added 0 filler inst  (cell FILL64 / prefix FILLER).
*INFO:   Added 2 filler insts (cell FILL32 / prefix FILLER).
*INFO:   Added 2 filler insts (cell GDCAP4 / prefix FILLER).
*INFO:   Added 2 filler insts (cell FILL16 / prefix FILLER).
*INFO:   Added 8 filler insts (cell FILL8 / prefix FILLER).
*INFO:   Added 7 filler insts (cell FILL4 / prefix FILLER).
*INFO:   Added 5 filler insts (cell FILL2 / prefix FILLER).
*INFO:   Added 3 filler insts (cell FILL1 / prefix FILLER).
*INFO: Total 29 filler insts added - prefix FILLER (CPU: 0:00:00.0).
```

After filler cells placement, Layout looks like this:



Run DRC again after filler cells placement. Violations should be 0.

```
 VERIFY DRC ..... Starting Verification
 VERIFY DRC ..... Initializing
 VERIFY DRC ..... Deleting Existing Violations
 VERIFY DRC ..... Creating Sub-Areas
 VERIFY DRC ..... Using new threading
 VERIFY DRC ..... Sub-Area: {0.000 0.000 38.000 38.000} 1 of 1
 VERIFY DRC ..... Sub-Area : 1 complete 0 Viols.

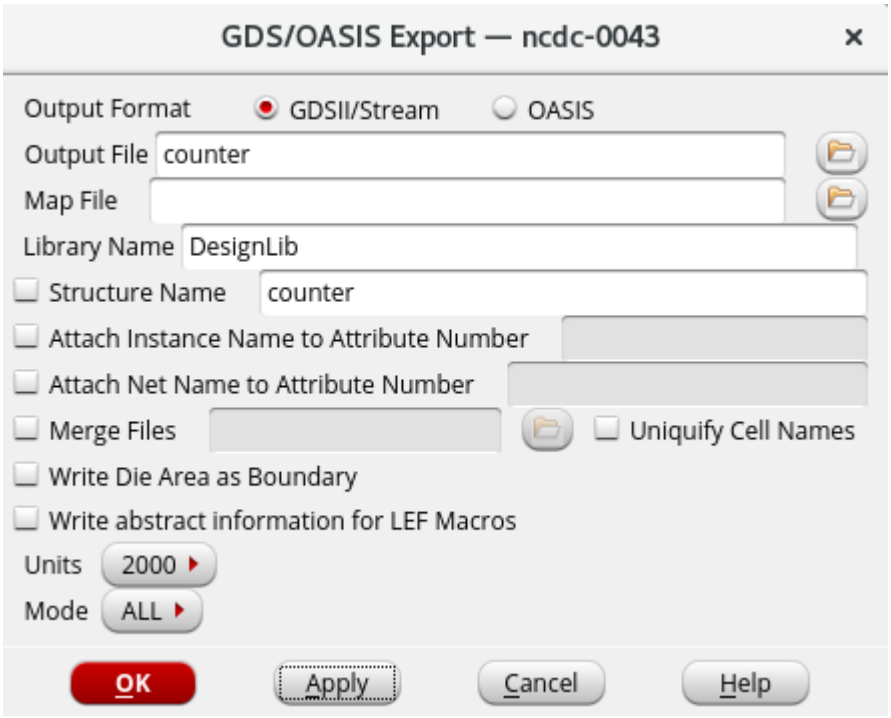
 Verification Complete : 0 Viols.

*** End Verify DRC (CPU: 0:00:00.0  ELAPSED TIME: 0.00  MEM: 256.1M) ***
```

Generating GDSII (Graphic Data System version II):

GDSII is the final output file format used in the VLSI design flow for sending a chip layout to fabrication. It contains all the physical layout information needed by the foundry to manufacture the chip.

Go to file -> save -> GDS/OASIS.



```
Object                                  Count
----------------------------------------
Instances                                  39

Ports/Pins                                  6
    metal layer M1                          6

Nets                                      118
    metal layer M1                         28
    metal layer M2                         64
    metal layer M3                         25
    metal layer M4                          1

    Via Instances                          72

Special Nets                               37
    metal layer M1                         21
    metal layer M4                          4
    metal layer M5                          8
    metal layer M6                          4

    Via Instances                          58

Metal Fills                                 0

    Via Instances                           0

Metal FillOPCs                              0

    Via Instances                           0

Metal FillDRCs                              0

    Via Instances                           0

Text                                       21
    metal layer M1                          8
    metal layer M2                          8
    metal layer M3                          3
    metal layer M4                          2


Blockages                                   0


Custom Text                                 0


Custom Box                                  0

Trim Metal                                  0

######Streamout is finished!
innovus 14>
```

Comment:

This output shows that the final GDSII file generation (streamout) was successfully completed in Innovus. It lists all metal layers (M1–M9) and via layers (VIA1–VIA8) used in the design, confirming that the layout data has been properly mapped for fabrication. The statistics show 43 standard cell instances, 131 nets routed across different metal layers, and 77 via instances connecting layers. It also includes 37 special nets like power and clock, but no metal fills or blockages were added, indicating a clean layout. Overall, this confirms that the design was correctly exported to the GDSII format for tape-out.

Critical Analysis:

In this lab, I learned the complete physical design flow up to the stream-out stage, where the final GDSII file is generated for fabrication. I understood how placement, clock tree synthesis, and routing directly affect DRC and timing results. During verification, I encountered a few DRC violations such as shorts and spacing issues, which helped me learn how to debug layout errors in Innovus using the GUI and reports. By analyzing the final stream-out report, I verified that the design had all required components—cells, pins, metals, and vias—properly implemented. Overall, this lab improved my understanding of real chip finishing steps and the importance of design rule checks before tape-out.